MATA KULIAH : **DESAIN DAN ANALISIS ALGORITMA**

DOSEN PENDAMPING : **DESI NURNANINGSIH, S.KOM., M.KOM.**

KELAS : **4D**

Kelompok 3 & 4 :

- **Fathurrachman (11180910000019)**
- **Ahmad Rusydi Said HRP (11180910000023)**
- **Toriq As Syarif (11180910000024)**
- **Mutia Salsabila (11180910000113)**
- **Nabila Lailika (11180910000128)**
- **Farid Dhiya Ul Arif (11180910000090)**
- **Rizka Amalia Putri (11180910000097)**
- **Dinda Difa Maharani (11180910000091)**
- **Muhammad Haidar Fahmi (11180910000100)**

REVIEW JURNAL

A Divide-And-Conquer Approach To The Pairwise Opposite Class-Nearest Neighbor (POC-NN) Algorithm.

Thanapant Raicharoen , Chidchanok Lursinsap

Advanced Virtual and Intelligent Computing Center (AVIC), Department of Mathematics, Faculty of Science, Chulalongkorn University, Bangkok 10330, Thailand

Received 20 June 2004; received in revised form 15 October 2004 Available online 17 February 2005 Communicated by R.P.W. Duin

---

Jurnal ini berisi metode baru berbasis *divide-and-conquer* terhadap pemilihan dan penempatan ulang pada set prototipe dari *training* set untuk aturan *nearest neighbor*, Yaitu analogi untuk membagi *training patterns* asli ke wilayah yang lebih kecil dengan menemukan prototype pasangan berlawanan terdekat (POC-NN) untuk wilayah, dan kemudian menggabungkan Prototipe POC-NN untuk wilayah menjadi satu set prototipe seleksi. Metode ini bertujuan mengurangi waktu komputasi dan ruang memori serta sensitivitas urutan dan noise data training. Set prototipe yang dikurangi berisi prototipe *Pairwise Opposite Class-Nearest Neighbor* (POC-NN) yang dekat dengan batasan keputusan dan digunakan sebagai pengganti pola *training*. Prototipe POC-NN diperoleh dengan pemisahan dan analisis berulang (iterasi) secara berulang-ulang (rekursif) dari data training menjadi dua wilayah sampai masing-masing daerah dikelompokkan dan diklasifikasikan dengan benar. Pemisahan ditentukan oleh prototipe POC-NN yang penting untuk menentukan lokasi semua hyperplanes yang terpisah. Jurnal ini disusun menjadi lima bagian. Bagian 2 dan 3 berisi metode POC-NN yang diusulkan untuk

prototype pemilihan dan penggantian, masing-masing, termasuk analisisnya. Di Bagian 4, beberapa percobaan dijelaskan dan kinerja metode ini di evaluasi. Bagian 5 kesimpulan makalah ini.

Bagian 2- Metodologi POC-NN untuk prototype pemilihan

Idenya adalah untuk mengisolasi subset dari set training yang memenuhi untuk prediksi tetangga terdekat, dan membuang data yang tersisa. Secara intuitif, itu tampaknya penting untuk menjaga titik *training* (pola) yang dekat dengan batas keputusan dan di sisi yang benar dari batas-batas itu, sementara beberapa titik yang jauh dari batas harus dibuang. Intinya dari ide ini adalah untuk menemukan pola POC-NN untuk dua data set kelas *training* yang diberikan dan kemudian pola yang tersisa bisa dibuang.

2.1 Algoritma mencari pola POC-NN

S adalah *training* set dari *n* pola, lalu dibagi menjadi 2 bagian S1 dan S2 dimana |S1|=n1 dan |S2|=n2. S1 dan S2 berada di kelas yang berbeda, tidak ada anggota yang beririsan. Algoritmanya sebagai berikut.

Fungsi FINDING-POC-NN (S:Dataset)

1. Biarkan S1 dan S2 menjadi dua set training dengan kelas 1 dan 2, yang ukurannya masing-masing adalah S1 dan S2.
2. **If** n1 ≥ n2

   Then
3. Xm = S1

   Biarkan xp2 ∈ S2 menjadi pola terdekat dengan xm

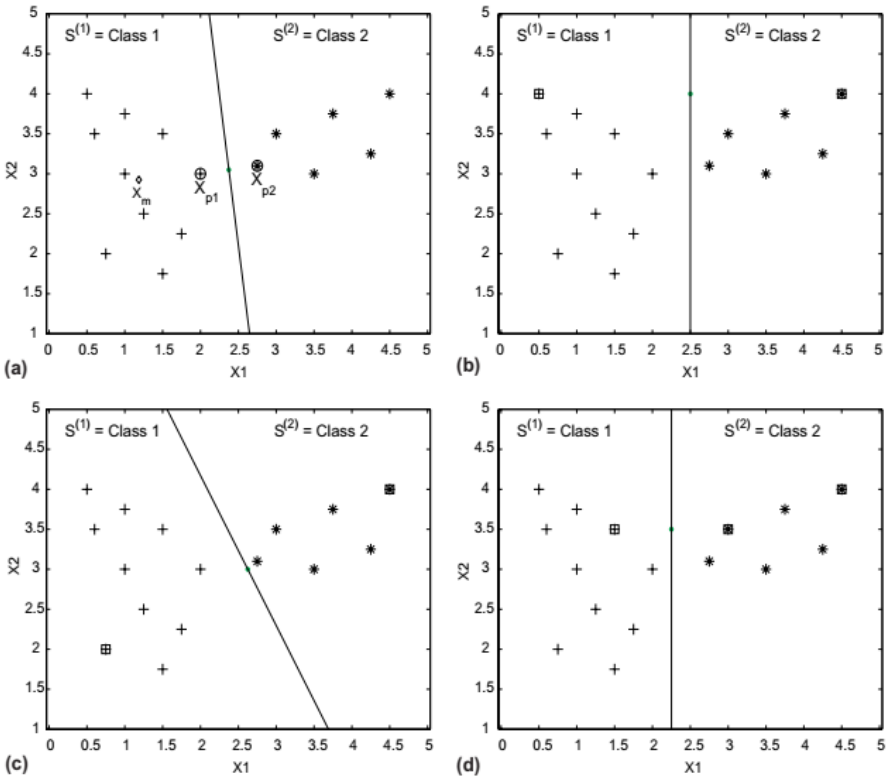
   Biarkan xp1 ∈ S1 menjadi pola terdekat dengan xp2

   Else
4. Xm=S2.

   Biarkan xp1 ∈ S2 menjadi pola terdekat dengan xm

   Biarkan xp2 ∈ S2 menjadi pola terdekat dengan xp1

   Endif
5. **Return** (xp1, xp2) sebagai pola POC-NN



2.2 Algoritma prototype pemilihan dengan POC-NN untuk masalah klasifikasi dua-kelas

Biarkan S menjadi set training n pola penyusunan dua kelas, dan POC-NN-SET awalnya sebagai set prototipe POC-NN kosong.

Function SELECTING-POC-NN (S: Dataset)

1. Cari POC-NN prototipe di S menggunakan

   (xp1, xp2) = FINDING-POC-NN (S).

2. Menentukan titik tengah $c = \frac{x_{p1} + x_{p1}}{2}$

3. Buat hyperplane pemisah H: {x|w . x – b = 0},

   Dimana $w = \frac{x_{p1} - x_{p2}}{||x_{p1} - x_{p2}||}$ dan b = w . c

4. **Save** (xp1, xp2) dan H yang terkait ke dalam POC-NN-SET.

5. Membagi semua pola S menjadi dua wilayah, yaitu R1 dan R2, di mana

   R1 = {xi ∈ S |w . xi – b ≥ 0}, dan

   R2 = {xi ∈ S |w . xi – b < 0} ∀ I, i = 1,…,n.

6. Temukan kesalahan klasifikasi di kedua wilayah.

7. **If** terdapat kesalahan klasifikasi di wilayah R1

   Then

8. Anggap semua data di wilayah R1 sebagai dataset baru

   Call SELECTING-POC-NN (R1).

   Endif

9. **If** terdapat kesalahan klasifikasi di wilayah R2

   Then

10. Anggap semua data di wilayah R2 sebagai dataset baru

    Call SELECTING-POC-NN (R2).

    Endif

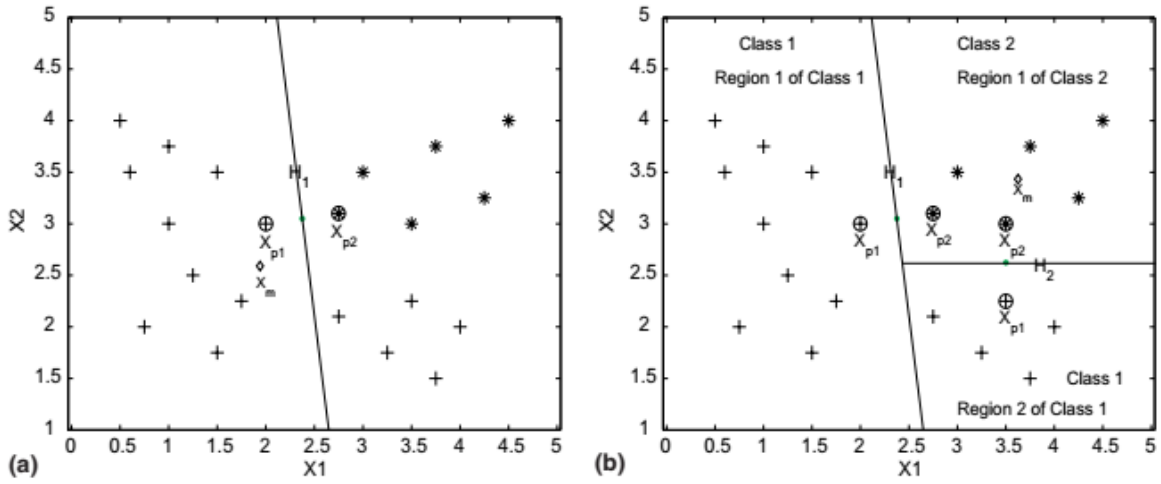
11. **If** tidak ada kesalahan klasifikasi lagi

    Then

12. **Return** POC-NN-SET sebagai seperangkat prototipe yang dipilih.

    Stop

    Endif



Bagian 3- Metodologi POC-NN untuk prototipe penggantian

Setelah melakukan pemilihan prototype oleh POC-NN algoritma pada suatu *training* set, *training set* tersebut dipisahkan menjadi beberapa wilayah dari kelas yang di klasifikasikan. Pada bagian ini, *training set* awal digantikan oleh sejumlah prototipe yang tidak bertepatan dengan pola didalam *training set*. Dalam metode ini mengganti prototype POC-NN disetiap wilayah dengan rata-rata pola di wilayahnya.

3.1 Algoritma prototipe penggantian dengan POC-NN untuk masalah klasifikasi 2 kelas

Memodifikasi prototipe pemilihan POC-NN untuk klasifikasi 2 kelas. Training set dipisahkan menjadi 3 wilayah dengan klasifikasi kelas yang benar, karena itu diperolehlah 3 prototipe MOR-NN (x_{mor}) yang digunakan untuk menggantikan 4 prototipe POC-NN sebelumnya.

3.2 Analisa algoritma penggantian POC-NN

Algoritma pemilihan POC-NN memiliki urutan yang independen, algoritma penggantian POC-NN hampir identik kecuali menghitung rata-rata untuk setiap wilayah. Selama operasi rata-rata memiliki urutan yang independen maka algoritma penggantian POC-NN ini juga independen

Fungsi SELECTING-POC-NN (S: Dataset)

1. Temukan prototipe POC-NN dalam S dengan menggunakan

   $x_{p1}, x_{p2} = FINDING - POC - NN(S)$

2. Tentukan titik pusat $c = \frac{x_{p1}+x_{p2}}{2}$

3. Buat hyperplane pemisah H:

   $\{x|w\,.x - b = 0\}$,

   dimana $w = \frac{x_{p1}-x_{p2}}{||x_{p1}-x_{p2||}}$ dan $b = w.c$

4. **Save** (x_{p1}, x_{p2}) dan sesuaikan H ke dalam *POC-NN-SET.*

5. Bagi semua pola S menjadi dua wilayah, yaitu R1 dan R2, di mana

   $R1 = \{x_i \in S|w.x_1 - b \geq 0$ dan

   $R2 = \{x_i \in S|w.x_1 - b < 0, \forall i, i = 1,2, \ldots., n$

6. *T*emukan kesalahan klasifikasi di kedua wilayah.

7. **If** ada kesalahan klasifikasi di wilayah R1 **Then**

8. Pertimbangkan semua data di wilayah ini R1 sebagai kumpulan data baru

   Call REPLACING-POC-NN (R1).

   Else

9. **Save** x_{mor} menjadi rata-rata R1 ke dalam *MOR-NN-SET.*

   Endif

10. **If** ada kesalahan klasifikasi di wilayah R2 **Then**

11. 11. Pertimbangkan semua data di wilayah ini R2 sebagai kumpulan data baru

    Call REPLACING-POC-NN (R2).

    Else

12. Save x_{mor} menjadi rata-rata R2 ke dalam *MOR-NN-SET.*

    Endif

13. **If** tidak ada lagi kesalahan klasifikasi

    Then

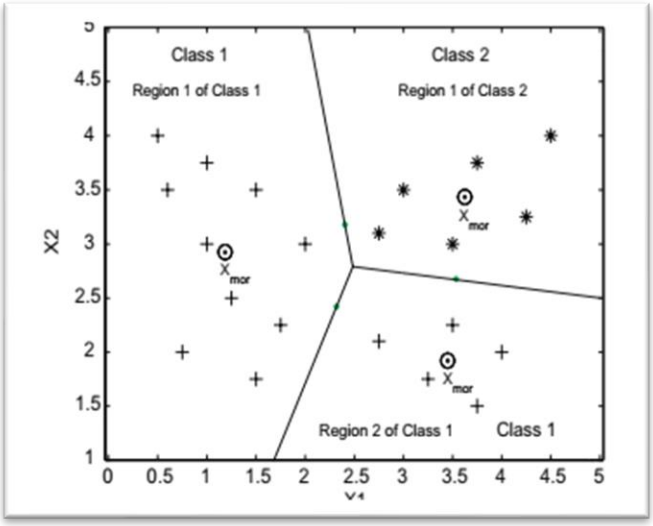
14. **Save** x_{mor} menjadi rata-rata R1 ke dalam *MOR-NN-SET*, dan

    Save x_{mor} menjadi rata-rata R2 ke dalam *MOR-NN-SET.*

15. **Return** *MOR-NN-SET* sebagai satu set prototipe pengganti.

    Stop

    Endif



Bagian 4- Hasil Percobaan POC-NN

Penulis menguji coba . Kumpulan data ini diambil dari Repositori UCI dari database pembelajaran mesin (Murphy dan Aha, 1994), data USPS (USPS, 1994) dan DNA dari Statlog (Michie et al., 1994). Dari hasil percobaan didapat bahwa POC-NN Algoritma memeliki waktu pelatihan terbaik untuk semua data set. Metode POC NN ini sangat kompetitif dalam Kompleksitas waktu dan ruang.

Source Code dalam Bahasa JAVA

```
import java.util.Scanner;
import java.lang.Math;

public class CariTitikTerdekat
{
   public static void main(String[] args)
   {
      Scanner scan = new Scanner(System.in);

      System.out.print("Masukkan interval titik sebaran: ");
      final int numtitik = scan.nextInt();

      double[][] titik = new double[numtitik][2];
      double shortestDistance=0;
      double distance=0;
      String titikdekat1="";
      String titikdekat2="";

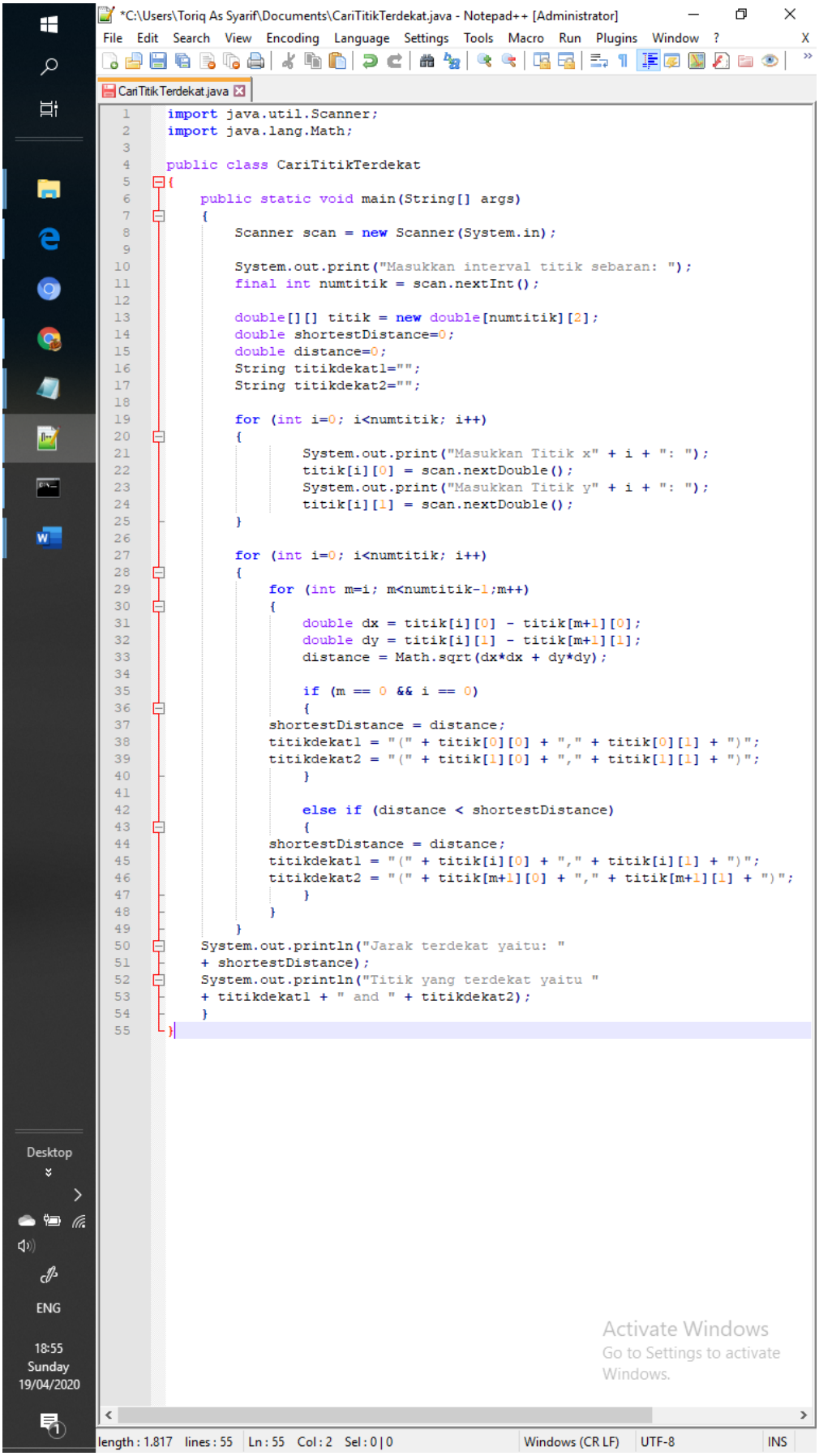
      for (int i=0; i<numtitik; i++)
      {
            System.out.print("Masukkan Titik x" + i + ": ");
            titik[i][0] = scan.nextDouble();
            System.out.print("Masukkan Titik y" + i + ": ");
            titik[i][1] = scan.nextDouble();
      }

      for (int i=0; i<numtitik; i++)
      {
         for (int m=i; m<numtitik-1;m++)
         {
            double dx = titik[i][0] - titik[m+1][0];
            double dy = titik[i][1] - titik[m+1][1];
            distance = Math.sqrt(dx*dx + dy*dy);

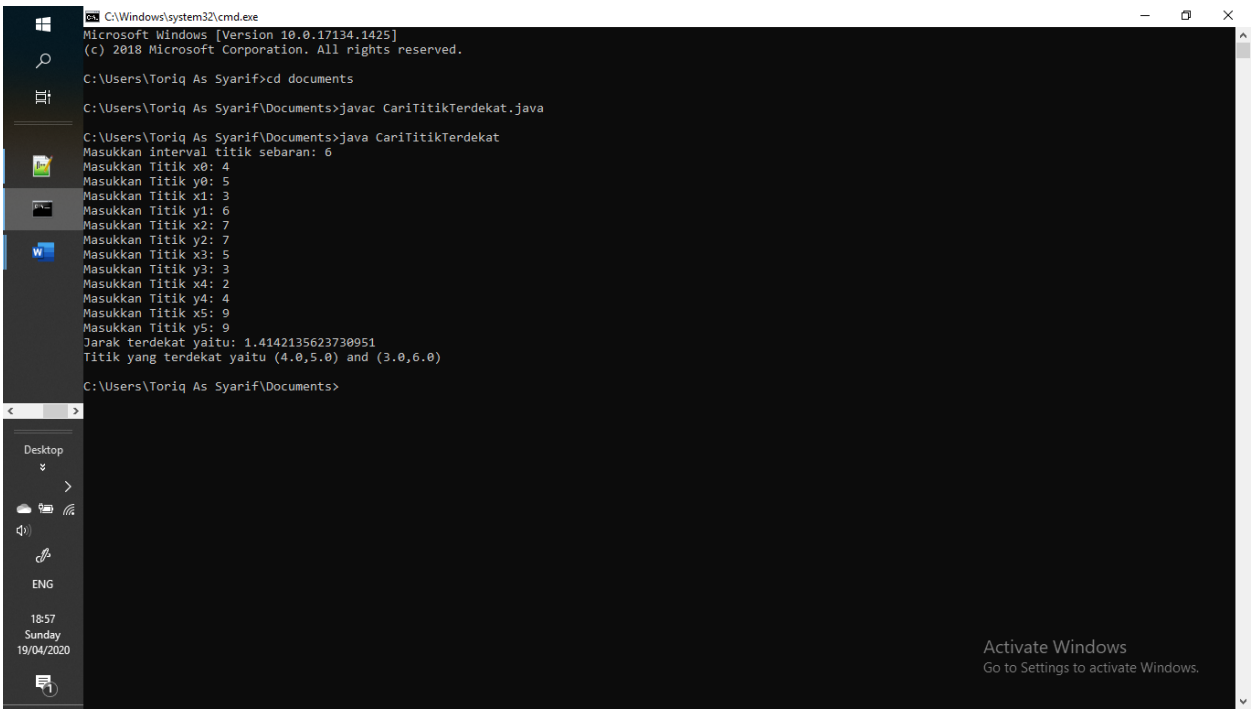
            if (m == 0 && i == 0)
            {
         shortestDistance = distance;
         titikdekat1 = "(" + titik[0][0] + "," + titik[0][1] + ")";
         titikdekat2 = "(" + titik[1][0] + "," + titik[1][1] + ")";
            }

            else if (distance < shortestDistance)
            {
         shortestDistance = distance;
         titikdekat1 = "(" + titik[i][0] + "," + titik[i][1] + ")";
```

```
            titikdekat2 = "(" + titik[m+1][0] + "," + titik[m+1][1] + ")";
                }
            }
        }
    System.out.println("Jarak terdekat yaitu: "
            + shortestDistance);
    System.out.println("Titik yang terdekat yaitu "
            + titikdekat1 + " and " + titikdekat2);
    }
}
```


*C:\Users\Toriq As Syarif\Documents\CariTitikTerdekat.java - Notepad++ [Administrator]
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
CariTitikTerdekat.java
import java.util.Scanner;
import java.lang.Math;
public class CariTitikTerdekat
{
public static void main(String[] args)
{
Scanner scan = new Scanner(System.in);
System.out.print("Masukkan interval titik sebaran: ");
final int numtitik = scan.nextInt();
double[][] titik = new double[numtitik][2];
double shortestDistance=0;
double distance=0;
String titikdekat1="";
String titikdekat2="";
for (int i=0; i<numtitik; i++)
{
System.out.print("Masukkan Titik x" + i + ": ");
titik[i][0] = scan.nextDouble();
System.out.print("Masukkan Titik y" + i + ": ");
titik[i][1] = scan.nextDouble();
}
for (int i=0; i<numtitik; i++)
{
for (int m=i; m<numtitik-1;m++)
{
double dx = titik[i][0] - titik[m+1][0];
double dy = titik[i][1] - titik[m+1][1];
distance = Math.sqrt(dx*dx + dy*dy);
if (m == 0 && i == 0)
{
shortestDistance = distance;
titikdekat1 = "(" + titik[0][0] + "," + titik[0][1] + ")";
titikdekat2 = "(" + titik[1][0] + "," + titik[1][1] + ")";
}
else if (distance < shortestDistance)
{
shortestDistance = distance;
titikdekat1 = "(" + titik[i][0] + "," + titik[i][1] + ")";
titikdekat2 = "(" + titik[m+1][0] + "," + titik[m+1][1] + ")";
}
}
}
System.out.println("Jarak terdekat yaitu: "
+ shortestDistance);
System.out.println("Titik yang terdekat yaitu "
+ titikdekat1 + " and " + titikdekat2);
}
}
Activate Windows
Go to Settings to activate Windows.
length : 1.817 lines : 55 Ln : 55 Col : 2 Sel : 0 | 0
Windows (CR LF) UTF-8 INS
Desktop
ENG
18:55
Sunday
19/04/2020

Output Program



KESIMPULAN

Metode POC-NN baru berdasarkan pendekatan divide-and-conquer untuk pemilihan dan penggantian satu set prototipe telah diusulkan. Untuk sejumlah data pelatihan yang terbatas, algoritma kami selalu menyatu dengan semua pola yang dipisahkan menjadi wilayah kelas yang diklasifikasikan dengan benar. Waktu kompleksitas yang diperlukan untuk menjalankan algoritma kami adalah O (dn^2m). Metode kami cepat, sesederhana, dan juga dapat mengatasi properti yang tidak diinginkan dari ketergantungan pesanan dan sensitivitas data yang berantakan. Prototype dan tingkat akurasinya juga dapat diatur oleh parameter yang ditentukan pengguna, disebut rasio interval penerimaan (αr). Tingkat prototipe dan akurasi yang diperoleh telah dibandingkan dengan CNN, Tomek, GKA, dan NN dengan semua pola pelatihan. Untuk sebagian besar dari hasil, metode kami menunjukkan kinerja yang lebih baik dalam prototipe dan tingkat akurasi dari metode lain. Hubungan antara αr dan jarak rata-rata di antara data juga variansnya tidak dipelajari di sini. Nilai αr harus adaptif secara lokal, sesuai dengan sifat distribusi data. Masalah ini sangat penting dan layak untuk diselidiki lebih lanjut.